Basic Algorithms

Abstract: *This chapter gives an overview of searching algorithms like linear search and binary search. It also consists of sorting algorithms like bubble sort, selection sort and insertion sort and Time complexity.*

Learning Objectives

- Introduction to searching algorithms
- Roots of Quadratic equation
- Introduction to sorting
- Time complexity

7.1. INTRODUCTION TO SEARCHING ALGORITHMS

DEFINITION

*A search **algorithm** is any **algorithm** which solves the search problem, viz., to retrieve information stored within some data structure, or calculated in the search space of a problem domain.*

Not even a single day pass, when we do not have to search for something in our day to day life, car keys, books, pen, mobile charger and what not. Same is the life of a computer, there is so much data stored in it, that whenever a user asks for some data, computer has to search its memory to look for the data and make it available to the user.

There are two popular algorithms available:

1. Linear Search
2. Binary Search

7.1.1. Linear Search

DEFINITION

Linear search, also known as sequential search, is a process that checks every element in the list sequentially until the desired element is found.

Linear search is a very basic and simple search algorithm. In Linear search, we search an element or value in a given array by traversing the array from the starting, till the desired element or value is found.

It compares the element to be searched with all the elements present in the array and when the element is **matched** successfully, it returns the index of the element in the array, else it returns-1.

Linear Search is applied on unsorted or unordered lists, when there are fewer elements in a list.

```
Step 1:  Start
Step 2 : input N
Step 3 : for I? 0 to N -1
Read A [ I ]
Step 4 : Loc = -1
Step 5 : For I = 0 to N- 1 do
Step 6 : If ( ele = A[I]) Then
Step 7:  Loc = I
Step 8 : Goto step 9
[ end if ]
[ end for ]
Step 9 : If ( Loc > = 0 ) then
Step 10: Print " ele Found in location " , Loc
Step 11: else
Step 12: Print " ele not found "
Step 13  Stop
```

Features of Linear Search Algorithm

1. It is used for unsorted and unordered small list of elements.
2. It has a time complexity of O(n), which means the time is linearly dependent on the number of elements, which is not bad, but not that good too.
3. It has a very simple implementation.

7.1.2. Binary Search

DEFINITION

***Binary** search, also known as half-interval search, logarithmic search, or binary chop, is a search **algorithm** that finds the position of a target value within a sorted array. **Binary** search compares the target value to the middle element of the array.*

Binary Search is used with sorted array or list. In binary search, we follow the following steps:

1. We start by comparing the element to be searched with the element in the middle of the list/ array.
2. If we get a match, we return the index of the middle element.

3. If we do not get a match, we check whether the element to be searched is less or greater than in value than the middle element.
4. If the element/number to be searched is greater in value than the middle number, then we pick the elements on the right side of the middle element(as the list/array is sorted, hence on the right, we will have all the numbers greater than the middle number), and start again from the step 1.
5. If the element/number to be searched is lesser in value than the middle number, then we pick the elements on the left side of the middle element, and start again from the step 1.

Binary Search is useful when there are large number of elements in an array and they are sorted. So a necessary condition for Binary search to work is that the list/array should be sorted.

This algorithm is used to search for an element in a sorted list. The value of the element in the middle of the list is compared with the value of the element to be searched for . If the middle element is larger, the desired element has to be in the upper side of the list. If the middle element is smaller, the desired element has to be in the lower half of the list. The number of elements to be searched is reduced by half in every iteration.

Algorithm for Binary Search:

```
Step 1: Low = 0
Step 2 : High = N - 1
Step 3 : Loc = -1
Step 4 : While ( Low < = High ) Do
Step 5 : M = ( Low + High ) / 2
Step 6 : If (ele = A[M ] ) Then
Step 7: Loc = M goto Step 12
[ end if ]
Step 8 : If (ele < A [M ] ) Then 10
Step 9: High ? M - 1
Step 10 : Else
Step 11 : Low ? M + 1
[ end of if ]
[ end of While loop ]
Step 12 : if ( Loc > = 0 ) Then
Step 13: Print " ele , found , search successful " , Loc
Step 14: else
Step 15 : Print " Item not found , search Unsuccessful "
[ end if ]
Step 16 : Stop
```

Features of Binary Search

1. It is great to search through large sorted arrays.
2. It has a time complexity of O(log n) which is a very good time complexity.
3. It has a simple implementation.

7.2. INTRODUCTION TO SORTING

DEFINITION

Arranging in an ordered sequence is called ***"sorting". Sorting*** *is a common operation in many applications, and efficient algorithms to perform it have been developed.*

Sorting is nothing but arranging the data in ascending or descending order. There are so many things in our real life that we need to search for, like a particular record in database, roll numbers in merit list, a particular telephone number in telephone directory, a particular page in a book etc. All this would have been a mess if the data had been kept unordered and unsorted, but fortunately the concept of **sorting** came into existence, making it easier for everyone to arrange data in an order, hence making it easier to search.

Sorting arranges data in a sequence which makes searching easier.

Sorting Efficiency

The two main criteria's to judge which algorithm is better than the other have been:

1. Time taken to sort the given data.
2. Memory Space required to do so.

Types of Sorting Algorithms

1. Bubble Sort
2. Insertion Sort
3. Selection Sort
4. Quick Sort
5. Merge Sort
6. Heap Sort

7.2.1. Bubble Sort Algorithm

Bubble Sort is a simple algorithm which is used to sort a given set of n elements provided in the form of an array with n number of elements. Bubble Sort compares all the elements one by one and sort them based on their values.

If the given array has to be sorted in ascending order, then bubble sort will start by comparing the first element of the array with the second element, if the first element is greater than the second element, it will swap both the elements, and then move on to compare the second and the third element, and so on.

If we have total n elements, then we need to repeat this process for n-1 times.

It is known as **bubble sort,** because with every complete iteration the largest element in the given array, bubbles up towards the last place or the highest index, just like a water bubble rises up to the water surface.

Sorting takes place by stepping through all the elements one-by-one and comparing it with the adjacent element and swapping them if required.

Bubble Sort Algorithm

Following are the steps involved in bubble sort(for sorting a given array in ascending order):

1. Starting with the first element(index = 0), compare the current element with the next element of the array.
2. If the current element is greater than the next element of the array, swap them.
3. If the current element is less than the next element, move to the next element. **Repeat Step 1.**

Algorithm for Bubble Sort

```
Step 1 : START
Step 2 : Input N
Step 3 : for I? 0 to N -1
Step 4 : Read ( A [ I ] )
( end of for loop )
Step 5 : for I? 1 to N -1
Step 6 : for J ? 0 to N -I - 1
Step 7 : If (A[ J ] > A [ J +1] )
Temp ? A [ J ]
A [ J ] ? A [ J + 1 ]
A [ J +1 ] ? Temp
[ end of if ]
[ end of J loop ]
[ end of I loop ]
Step 8 : for J ? 0 to N -1
Print A [ J ]
Step 9 : Stop
```

7.2.2. Selection Sort Algorithm

Selection sort is conceptually the simplest sorting algorithm. This algorithm will first find the **smallest** element in the array and swap it with the element in the **first** position, then it will find the **second smallest** element and swap it with the element in the second position, and it will keep on doing this until the entire array is sorted.

It is called selection sort because it repeatedly selects the next-smallest element and swaps it into the right place.

Following are the steps involved in selection sort(for sorting a given array in ascending order):

1. Starting from the first element, we search the smallest element in the array, and replace it with the element in the first position.
2. We then move on to the second position, and look for the smallest element present in the subarray, starting from index 1, till the last index.
3. We replace the element at the second position in the original array, or we can say at the first position in the subarray, with the second smallest element.

4. This is repeated, until the array is completely sorted.

Algorithm for selection sort:

```
Step 1 : START
Step 2 : Input N
Step 3 : for I? 0 to N -1
Step 4 : Read A [ I ]
Step 5 : for I ? 0 to N -2
Step 6 : S ? A [ I]
Step 7 : Pos ? I
Step 8 : for J ? I + 1 to N -1
Step 9 : If ( A[ J] < S )
Step 10 : S ? A [ J ]
Step 11 : Pos ? J
[ end of if ]
[ end of J for loop ]
Step 12 : A [ Pos ] ? a[ I ]
Step 14 : A[ I ] ? S
[end of I for loop ]
Step 15: for I = 0 to N - 1
Step 16: Print A[ I ]
Step 17: Stop
```

7.2.3. Insertion Sort Algorithm

Following are some of the important characteristics of **Insertion Sort:**

1. It is efficient for smaller data sets, but very inefficient for larger lists.
2. Insertion Sort is adaptive, that means it reduces its total number of steps if a partially sorted array is provided as input, making it efficient.
3. It is better than Selection Sort and Bubble Sort algorithms.
4. Its space complexity is less. Like bubble Sort, insertion sort also requires a single additional memory space.
5. It is a **stable** sorting technique, as it does not change the relative order of elements which are equal.

Algorithm to sort an element using Insertion Sort

```
Ans: Algorithm for Insertion sort
Step 1 : START
Step 2 : Input N
Step 3 : for I? 0 to N -1
Read A [ I ]
```

```
( end of I for loop )
Step 4 : For I ? 1 to N -1 Do
Step 5 : J = I
Step 6 : While ( J > = 1 ) and ( A[ J ] < A [ J -1 ] )
Step 7 : TEMP = A [ J ]
Step 8 : A [ J ] = A [ J - 1 ]
Step 9 : A[ J - 1 ] = TEMP
Step 10 : J = J - 1
[ end of While loop ]
[ end of step I For loop ]
Step 11 : for I ? 0 to N -1
Step 12: Print A [ I ]
Step 13 : Stop
```

7.3. ROOTS OF QUADRATIC EQUATION

Algorithm:

```
1) Start
2) Declare the required variables
3) Read the coefficients a,b,c of quadratic equation
4) Calculate d=b2-4ac
5) If d<0,display" roots are imaginary" & goto step 6
else
calculate x=(-b+ sqrt(d))/2a
y =(-b-sqrt(d))/2a
6) Display real roots x,y
7) Stop
```

7.4. TIME COMPLEXITY

DEFINITION

***Time complexity** is the **computational complexity** that describes the amount of time it takes to run an algorithm.*

Imagine a classroom of 100 students in which you gave your pen to one person. Now, you want that pen. Here are some ways to find the pen and what the O order is.

O(n²) : You go and ask the first person of the class, if he has the pen. Also, you ask this person about other 99 students in the classroom if they have that pen & So on, This is called $O(n^2)$.

O(n): Going and asking each student individually is O(N).

O(log *n*): Now you divide the class into two groups, then ask: "Is it on the left side, or the right side of the classroom?" Then you take that group and divide it into two and ask again, and so on. Repeat the process till you are left with one student who has your pen. This is called O(log *n*).

CONCLUDING REMARKS

In this chapter, you have studied an overview of searching algorithms like linear search and